

Explain Correctness, Complexity, and Tests

A complete coding answer includes a contract, a reason the algorithm is correct, an implementation, tests, and a realistic cost analysis. These are not optional finishing touches. They expose assumptions early and help you recover from feedback without replacing the whole solution blindly.

For: Candidates who often reach a plausible implementation but struggle to justify it, test systematically, or give complexity that matches the actual code.

Suggested pace: 2 weeks, then use in every coding session | All DSA levels

What you will be able to do

- Turn an ambiguous prompt into explicit input and output assumptions.
- Use invariants, induction, or exchange reasoning at an appropriate level.
- Design tests from failure modes rather than random happy-path examples.
- Include library operations, copies, recursion, and output size in complexity.

Build the right mental model

Clarification is part of solving

Ask about duplicates, sortedness, mutation, missing answers, size limits, and return format when relevant. Do not ask every possible question mechanically; resolve the assumptions that change the algorithm or its safety.

An invariant connects iterations

State what is true before and after each step, why initialization establishes it, why the update preserves it, and why termination yields the answer. Recursive solutions use an analogous subproblem contract. The explanation should match the code, not a different ideal algorithm.

Tests discriminate hypotheses

A useful test targets a boundary, duplicate, stale state, overflow, or invalid assumption. Tiny counterexamples are easier to trace. Compare with an independent brute-force oracle on small random inputs when the problem permits it.

Study with evidence.

Reserve a fixed portion of each practice session for explanation and tests. After coding, read the implementation as if reviewing someone else's work: identify hidden copies, stale variables, mutation, and failure paths before reporting a complexity class.

Days 1-3: Contract and baseline

Rewrite several prompts as concise contracts. Implement or describe a simple correct baseline and identify its repeated work. State which constraints justify an optimization.

Deliverable and check: A contract and baseline for each problem, including no-answer and minimal-input behavior.

Days 4-6: Correctness arguments

Practice one loop invariant, one recursive induction, and one greedy exchange argument. Align variables in the explanation with the implementation. Ask a peer to find an update that might break the invariant.

Deliverable and check: A short proof sketch that survives a changed example and does not merely restate the code.

Days 7-10: Adversarial tests

Build tests for empty/minimal input where allowed, duplicates, ties, monotone extremes, and absent answers. Use exhaustive or randomized small cases against a baseline to catch overlooked combinations.

Deliverable and check: At least one regression case that fails the buggy version and passes the corrected one.

Days 11-14: Cost and communication

Audit loops, recursion depth, sorting, substring creation, hash operations, and output size. Run a timed mock with an explicit testing segment. Explain a bug correction through the broken invariant rather than apologizing vaguely.

Deliverable and check: A complete solution narrative with justified costs and a reproducible set of checks.

A repeatable practice session

Spend about 15 minutes recalling the previous topic without notes, 45 minutes on today's problem or lab, and 15 minutes explaining and checking the result. Record the failed assumption, not just the final answer. Revisit weak material after a few days and again the following week.

Apply it. Explain it.

Use a brute-force oracle to test an optimized method

You optimize a contiguous-range calculation from quadratic time to a linear scan. The examples pass, but an update-order bug may remain. Small inputs make exhaustive comparison practical.

A worked approach

1. Write a straightforward baseline that enumerates every valid range and computes the requested result directly. Keep it structurally different from the optimized code so both are less likely to share the same mistake.
2. Generate short arrays from a small value set, including zero, duplicates, and negative values when allowed. Compare outputs for each array and target. Stop at the first mismatch and shrink it to the smallest explanatory input.
3. Trace the optimized state on that input, identify the violated invariant, and add a fixed regression case before correcting the implementation. Rerun the same exhaustive comparison and neighboring boundaries afterward.

Expected result

The oracle supplies evidence against implementation defects while the invariant explains general correctness. Exhaustive small tests do not prove every large input, but they are highly effective at finding boundary and state-update errors.

Verify the result

- Does the baseline implement the same output contract?
- Are generated inputs restricted to the allowed domain?
- Is the baseline independent enough to avoid reproducing the optimized bug?

Extend the exercise

Add a performance comparison only after outputs agree. Use increasing sizes and record the workload; do not infer asymptotic behavior from one tiny example or ignore the cost of generating the input.

Make the learning stick.

Can tests replace a correctness proof?

No finite ordinary test set covers every possible input, but tests are essential for validating implementation and assumptions. A practical interview answer combines a concise proof argument with targeted tests rather than choosing one or the other.

Should output memory count as space?

State the convention. Report auxiliary working memory separately from output when useful, but do not hide a large output or copied input. Recursion stacks and library-created temporary objects are also real space costs.

What should I say when a test reveals a bug?

Explain the failing case, identify the broken invariant, make the smallest grounded correction, and rerun the case plus related boundaries. This demonstrates precise debugging and is better than defending the incorrect result.

Avoid these traps

- Reporting complexity for an idea rather than the actual implementation.
- Testing only the examples supplied in the prompt.
- Using a baseline that shares the same optimization and bug.

Ready for the next step?

- Input and output assumptions are explicit.
- The correctness argument matches the implementation.
- Tests target meaningful failure modes and boundaries.
- Complexity includes copies, recursion, and output conventions.

Reliable next reads

- [MIT algorithms course](#)
- [Python unittest](#)
- [Hypothesis property-based testing](#)

An independent preparation guide, not an employer-endorsed syllabus or a guarantee of an offer. Company examples are practice contexts, not verified past interview questions. Use only public, synthetic, or authorized material in exercises.